☕ Guia Lúdico da API java.time

Oct 27, 2026 · @Jader

🏙️ Bem-vindo à Cidade do Tempo

Desde o Java 8, datas e horas moram no pacote `java.time` — uma cidade planejada que substituiu o velho bairro caótico de `Date` e `Calendar`. Três leis regem a cidade:

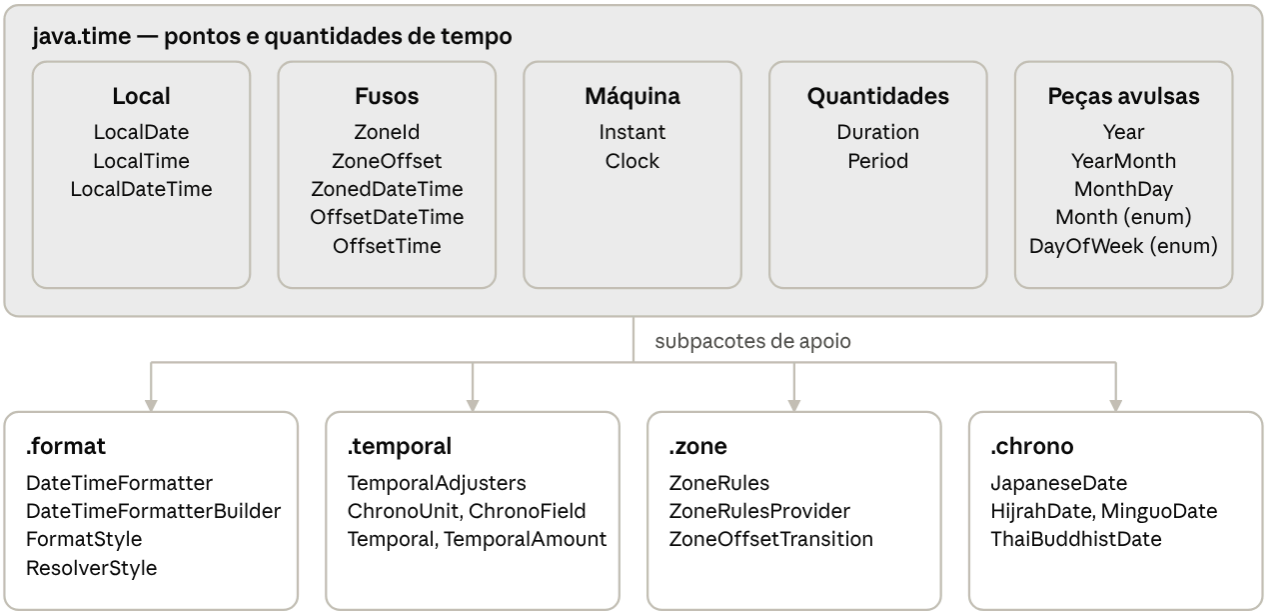
1. **Imutabilidade** 🧊 — nenhum objeto muda. `data.plusDays(1)` devolve um objeto **novo**; o original fica intacto. Por isso são *thread-safe*.
2. **Nada de construtor** 🚪 — você nunca escreve `new LocalDate(...)`. Entra-se pelas portas de fábrica: `now()`, `of()`, `parse()`, `from()`.
3. **Nomes que falam** 🗣️ — `plus`, `minus`, `with`, `is`, `get`, `to`, `at` funcionam igual em quase todas as classes.

Como ler este guia: cada bairro traz um mascote, a utilidade principal de cada classe e um trecho de código pronto para colar.

Prefixo	Faz o quê	Exemplo
`of`	cria a partir de partes	`LocalDate.of(2026, 10, 27)`
`now`	pega o agora do relógio	`LocalTime.now()`
`parse`	lê de um texto	`LocalDate.parse("2026-10-27")`
`plus` / `minus`	soma ou subtrai	`data.plusWeeks(2)`
`with`	troca um pedaço	`data.withDayOfMonth(1)`
`get`	lê um pedaço	`data.getMonthValue()`
`is`	pergunta algo	`data.isLeapYear()`
`at`	junta com outra peça	`data.atTime(14, 30)`
`to`	converte para outro tipo	`dataHora.toLocalDate()`
`format`	vira texto	`data.format(fmt)`

🗺️ Mapa da Cidade — a estrutura da API

java.time: cinco bairros no pacote principal e quatro subpacotes de apoio



mapa da API java.time · 5 bairros + 4 subpacotes

O pacote principal `java.time` guarda as classes que você usa todo dia; os subpacotes `format`, `temporal`, `zone` e `chrono` são a infraestrutura que as faz funcionar. Cada bairro tem sua seção abaixo.

🏡 Bairro Local — a hora "da parede"

Aqui mora o tempo **sem fuso**: o que está escrito no calendário da cozinha ou no relógio da sala. Serve para aniversários, vencimentos, horário de expediente — tudo que não precisa saber se é Brasília ou Tóquio.

Classe	Mascote	Guarda	Use quando…
`LocalDate`	📅 o Calendário	ano-mês-dia	só a data importa (nascimento, vencimento de boleto)
`LocalTime`	⏰ o Despertador	hora:min:seg.nano	só o horário importa (abertura da loja às 09:00)
`LocalDateTime`	🗓️⏰ a Agenda	data + hora	um compromisso local, sem viajar de fuso

```
LocalDate hoje       = LocalDate.now();
LocalDate natal      = LocalDate.of(2026, Month.DECEMBER, 25);
LocalDate lido       = LocalDate.parse("2026-10-27");     // ISO: yyyy-MM-dd

LocalTime almoco     = LocalTime.of(12, 30);
LocalTime depois     = almoco.plusMinutes(45);           // 13:15

LocalDateTime reuniao = LocalDateTime.of(2026, 11, 3, 14, 0);
LocalDateTime junto   = natal.atTime(almoco);            // data + hora
LocalDate soData      = reuniao.toLocalDate();           // separa de novo

// Perguntas úteis
hoje.isBefore(natal);        // true
hoje.isLeapYear();           // ano bissexto?
hoje.lengthOfMonth();        // 28, 29, 30 ou 31
hoje.getDayOfWeek();         // TUESDAY
```

> 💡 **Regra de bolso:** se a pergunta é "que dia/hora está escrito?", é Local. Se é "que instante aconteceu no planeta?", vá para os Fusos ou para a Sala de Máquinas.

🌍 Bairro dos Fusos — o tempo que viaja

Quando o sistema atende gente em lugares diferentes (ou o horário de verão pode voltar), o tempo precisa carregar **onde** ele vale.

Classe	Mascote	Guarda	Use quando…
`ZoneId`	🗺️ o Mapa-múndi	uma região com regras (`America/Sao_Paulo`)	precisa das regras de horário de verão da região
`ZoneOffset`	📏 a Régua	um deslocamento fixo (`-03:00`)	só importa a diferença para UTC, sem regras
`ZonedDateTime`	✈️ o Viajante	data + hora + `ZoneId`	agendar algo "às 9h de São Paulo", inclusive nos anos com horário de verão
`OffsetDateTime`	📨 o Carteiro	data + hora + `ZoneOffset`	gravar em banco, trafegar em API/JSON (ISO-8601 com offset)

Classe	Mascote	Guarda	Use quando…
`OffsetTime`	⏱️ o Primo raro	hora + `ZoneOffset`	horário diário com offset fixo (pouco usado)

```
ZoneId sp    = ZoneId.of("America/Sao_Paulo");
ZoneId tokyo = ZoneId.of("Asia/Tokyo");

ZonedDateTime aquiAgora = ZonedDateTime.now(sp);
ZonedDateTime emTokyo   = aquiAgora.withZoneSameInstant(tokyo);  // mesmo
instante, outro relógio
ZonedDateTime mesmaHora = aquiAgora.withZoneSameLocal(tokyo);    // mesmo
relógio, outro instante!

OffsetDateTime paraApi = OffsetDateTime.of(
        LocalDateTime.of(2026, 10, 27, 16, 20), ZoneOffset.ofHours(-3));
// 2026-10-27T16:20-03:00

ZoneId.getAvailableZoneIds();   // todas as regiões conhecidas
```

> ⚠️ **Pegadinha clássica:** `withZoneSameInstant` converte (16h em SP = 04h do dia seguinte em Tóquio). `withZoneSameLocal` só troca a etiqueta e muda o instante real.

> 💡 Prefira `ZoneId` com nome de região a offsets fixos: o Brasil já teve, aboliu e pode voltar a ter horário de verão — a região sabe disso, o offset não.

⚙️ Sala de Máquinas — o tempo do computador

Para a máquina, o tempo é só uma contagem de segundos desde 1970-01-01T00:00Z (a *época Unix*). Sem calendário, sem fuso, sem opinião.

Classe	Mascote	Guarda	Use quando…
`Instant`	🤖 o Robô	segundos + nanos desde a época, em UTC	logs, auditoria, timestamps, medir "quando aconteceu" no planeta
`Clock`	🕰️ o Relógio-mestre	de onde vem o "agora" + um fuso	injetar o tempo para testar código que usa `now()`

```
Instant agora   = Instant.now();                // 2026-10-27T19:20:00Z
long epochMs    = agora.toEpochMilli();
Instant doEpoch = Instant.ofEpochSecond(1_700_000_000);

// Instant → hora humana: precisa dizer o fuso
ZonedDateTime local = agora.atZone(ZoneId.of("America/Sao_Paulo"));

// Clock: o segredo dos testes 🧪
Clock fixo = Clock.fixed(Instant.parse("2026-01-01T00:00:00Z"),
ZoneOffset.UTC);
LocalDate hojeNoTeste = LocalDate.now(fixo);     // sempre 2026-01-01
```

> 💡 Guardou um momento no banco ou num log? Use `Instant` (ou `OffsetDateTime`). Vai mostrar para gente? Converta para o fuso de quem lê só na hora de exibir.

📦 Armazém das Quantidades — "quanto tempo?"

Até aqui vimos **pontos** no tempo. No armazém ficam as **distâncias** entre pontos.

Classe	Mascote	Mede em	Use quando…
`Duration`	⏳ a Ampulheta	segundos + nanos (visão de máquina)	tempo de execução, timeout, "2h30min de reunião"
`Period`	🎂 o Bolo de aniversário	anos + meses + dias (visão de calendário)	idade, prazo contratual, "3 meses de garantia"
`ChronoUnit`	📏 a Fita métrica	uma unidade (`DAYS` , `HOURS` , `MONTHS` …)	contar "quantos X entre A e B" num número só

```
// Duration: entre horários ou instantes
Duration d = Duration.between(LocalTime.of(9, 0), LocalTime.of(17, 30));
d.toHours();      // 8
d.toMinutes();    // 510
Duration.ofMinutes(90);   // PT1H30M

// Period: entre datas
Period idade = Period.between(LocalDate.of(1990, 5, 10), LocalDate.now());
idade.getYears(); // anos completos
Period.ofMonths(3);       // P3M

// ChronoUnit: um número total
long dias = ChronoUnit.DAYS.between(LocalDate.of(2026, 1, 1),
LocalDate.of(2026, 12, 25)); // 358
LocalDate prox = LocalDate.now().plus(2, ChronoUnit.WEEKS);
```

> ⚠️ `Period.between(...).getDays()` **não** é o total de dias — é só a sobra depois de anos e meses. Para o total, use `ChronoUnit.DAYS.between(...)`.

> 🧠 Por que dois tipos? Um "dia" no calendário pode ter 23 ou 25 horas quando o horário de verão muda. `Period` soma "1 dia de calendário"; `Duration` soma exatamente 24h.

🧩 Peças Avulsas — pedaços de calendário

Nem sempre você precisa da data inteira. Estas peças guardam só o pedaço que interessa — e deixam o código dizer exatamente o que quer.

Classe	Tipo	Guarda	Exemplo de uso
`Year`	classe	só o ano	`Year.of(2028).isLeap()` → true
`YearMonth`	classe	ano + mês	validade de cartão, competência de folha: `YearMonth.of(2026, 10).atEndOfMonth()`
`MonthDay`	classe	mês + dia	aniversário ou feriado fixo: `MonthDay.of(12, 25)`
`Month`	enum 🎲	JANUARY… DECEMBER	`Month.FEBRUARY.length(true)` → 29
`DayOfWeek`	enum 🎲	MONDAY… SUNDAY	`hoje.getDayOfWeek() == DayOfWeek.SATURDAY`

```
YearMonth competencia = YearMonth.now();          // 2026-10
LocalDate ultimoDia   = competencia.atEndOfMonth(); // 2026-10-31

MonthDay aniversario = MonthDay.of(Month.MAY, 10);
boolean ehHoje = aniversario.equals(MonthDay.now());

DayOfWeek d = DayOfWeek.FRIDAY.plus(3);           // MONDAY (enum dá a volta)
d.getDisplayName(TextStyle.FULL, Locale.of("pt", "BR")); // "segunda-feira"
```

> 💡 `Month` e `DayOfWeek` são enums: compare com `==`, use em `switch` e nunca mais erre o "mês 0 = janeiro" do `Calendar`. Aqui janeiro é 1. 🎉

🔧 Oficina de Ajustes — `java.time.temporal`

Os bastidores da cidade. Você raramente instancia estas peças, mas elas são as ferramentas que fazem `plus`, `with` e `get` funcionarem em todas as classes.

Peça	Mascote	Utilidade principal
`Temporal` / `TemporalAccessor`	🔌 a Tomada universal	interfaces que todas as classes de data/hora implementam — permitem métodos genéricos
`TemporalAdjuster`	🪄 a Varinha	regra que transforma uma data em outra
`TemporalAdjusters`	🧰 a Caixa de varinhas	ajustes prontos: primeiro/último dia, próxima sexta…

Peça	Mascote	Utilidade principal
`ChronoField`	🏷️ as Etiquetas	nomeia cada campo (`DAY_OF_MONTH`, `HOUR_OF_DAY`…) para ler ou trocar
`ChronoUnit`	📏 a Fita métrica	unidades para somar e medir (vista no Armazém)
`TemporalAmount`	📦 a Caixa de quantidade	interface comum de `Duration` e `Period`

```
import static java.time.temporal.TemporalAdjusters.*;

LocalDate hoje = LocalDate.of(2026, 10, 27);

hoje.with(firstDayOfMonth());                 // 2026-10-01
hoje.with(lastDayOfMonth());                  // 2026-10-31
hoje.with(next(DayOfWeek.FRIDAY));            // 2026-10-30
hoje.with(firstInMonth(DayOfWeek.MONDAY));    // 2026-10-05
hoje.with(firstDayOfNextYear());              // 2027-01-01

hoje.get(ChronoField.DAY_OF_YEAR);            // 300
hoje.with(ChronoField.MONTH_OF_YEAR, 12);     // 2026-12-27

// Sua própria varinha: próximo dia útil 🪄
TemporalAdjuster proximoDiaUtil = t -> {
    LocalDate d = LocalDate.from(t).plusDays(1);
    while (d.getDayOfWeek() == DayOfWeek.SATURDAY || d.getDayOfWeek() ==
DayOfWeek.SUNDAY) {
        d = d.plusDays(1);
    }
    return t.with(d);
};
hoje.with(proximoDiaUtil);                    // 2026-10-28
```

🎨 Ateliê DateTimeFormatter — o tradutor oficial

`DateTimeFormatter` (pacote `java.time.format`) é o intérprete da cidade: transforma objeto em texto (**format**) e texto em objeto (**parse**). Ele é imutável e *thread-safe* — crie uma vez como `static final` e reutilize à vontade (bem diferente do antigo `SimpleDateFormat`, que quebrava em paralelo 💥).

🔄 Ida e volta

```
DateTimeFormatter BR = DateTimeFormatter.ofPattern("dd/MM/yyyy");

String texto   = LocalDate.of(2026, 10, 27).format(BR);   // "27/10/2026"
(objeto → texto)
LocalDate data = LocalDate.parse("27/10/2026", BR);       // 2026-10-27
(texto → objeto)
```

🏭 Três jeitos de conseguir um formatter

Jeito	Quando usar	Exemplo
Constantes prontas	APIs, JSON, logs, troca entre sistemas	`DateTimeFormatter.ISO_LOCAL_DATE`
`ofPattern("...")`	formato sob medida para telas e relatórios	`ofPattern("dd/MM/yyyy HH:mm")`
`ofLocalizedDate(FormatStyle.X)`	deixar o idioma/país decidir o formato	`ofLocalizedDate(FormatStyle.LONG)`

📚 Constantes prontas mais usadas

Constante	Saída de exemplo
`ISO_LOCAL_DATE`	`2026-10-27`
`ISO_LOCAL_TIME`	`16:20:00`
`ISO_LOCAL_DATE_TIME`	`2026-10-27T16:20:00`
`ISO_OFFSET_DATE_TIME`	`2026-10-27T16:20:00-03:00`
`ISO_ZONED_DATE_TIME`	`2026-10-27T16:20:00-03:00[America/Sao_Paulo]`
`ISO_INSTANT`	`2026-10-27T19:20:00Z`

Constante	Saída de exemplo
`BASIC_ISO_DATE`	`20261027`
`RFC_1123_DATE_TIME`	`Tue, 27 Oct 2026 16:20:00 -0300`

> 💡 O `toString()` e o `parse(texto)` sem formatter de todas as classes já usam o padrão ISO. Por isso `LocalDate.parse("2026-10-27")` funciona sem nada extra.

🔤 O alfabeto do `ofPattern`

A quantidade de letras muda o resultado: mais letras = forma mais longa.

Letra	Significa	Padrão → saída (27/10/2026 16:05:09, pt-BR)
`d`	dia do mês	`d` → 27 · `dd` → 27 (e 05 para dia 5)
`M`	mês	`M` → 10 · `MM` → 10 · `MMM` → out. · `MMMM` → outubro
`y` / `u`	ano (`y` = ano da era, `u` = ano proleptico)	`yy` → 26 · `yyyy` → 2026
`E`	dia da semana (texto)	`E` → ter. · `EEEE` → terça-feira
`H`	hora 0–23	`HH` → 16
`h`	hora 1–12 (use com `a`)	`hh` → 04
`a`	AM/PM	`a` → PM
`m`	minuto	`mm` → 05
`s`	segundo	`ss` → 09
`S`	fração de segundo	`SSS` → milissegundos
`n`	nanossegundo do segundo	`n` → 0
`D`	dia do ano	`D` → 300
`w`	semana do ano (depende do Locale)	`w` → 44
`Q`	trimestre	`Q` → 4 · `QQQQ` → 4º trimestre
`VV`	id do fuso	`VV` → America/Sao_Paulo

Letra	Significa	Padrão → saída (27/10/2026 16:05:09, pt-BR)
`z`	nome do fuso	`z` → BRT
`XXX` / `xxx`	offset	`XXX` → -03:00 (`Z` quando for UTC)
`'texto'`	texto literal	`'às'` → às
`''`	uma aspa simples	`''` → '
`[ ]`	trecho opcional (ótimo no parse)	`HH:mm[:ss]` aceita com ou sem segundos

> 🚨 **As três pegadinhas mais famosas:**
>
> 1. `MM` é mês, `mm` é minuto. Trocar os dois gera datas como "27/05/2026" quando deveria ser outubro.
> 2. `HH` é 0–23; `hh` é 1–12 e sem `a` vira confusão entre 4h e 16h.
> 3. `YYYY` (maiúsculo) é o **ano-semana**: em 29/12/2025 ele pode mostrar 2026. Para ano normal, use `yyyy` ou `uuuu`.

🌎 Idioma e estilo com `Locale`

```
Locale ptBR = Locale.of("pt", "BR");   // Java 19+; antes: new Locale("pt", "BR")
LocalDateTime dt = LocalDateTime.of(2026, 10, 27, 16, 5);

DateTimeFormatter longo = DateTimeFormatter
        .ofPattern("EEEE, d 'de' MMMM 'de' yyyy 'às' HH'h'mm", ptBR);
dt.format(longo);   // "terça-feira, 27 de outubro de 2026 às 16h05"

DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).withLocale(ptBR).format(dt);  // 27/10/2026
DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).withLocale(ptBR).format(dt);   // 27 de outubro de 2026
DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL).withLocale(ptBR).format(dt);   // terça-feira, 27 de outubro de 2026
```

> ⚙️ Sem `Locale`, o formatter usa o idioma padrão da JVM. No servidor em inglês, `MMMM` vira "October". Em sistemas que imprimem nomes de mês ou dia, passe sempre o `Locale`.

🛡️ Parse rigoroso e tratamento de erro

```
DateTimeFormatter rigoroso = DateTimeFormatter.ofPattern("dd/MM/uuuu")
        .withResolverStyle(ResolverStyle.STRICT);

try {
    LocalDate.parse("31/02/2026", rigoroso);   // 31 de fevereiro? 🚫
} catch (DateTimeParseException e) {
    System.out.println("Data inválida: " + e.getParsedString());
}
```

`ResolverStyle`	Com "31/02/2026"
`STRICT`	lança exceção (com `STRICT`, use `uuuu`; `yyyy` exige era)
`SMART` (padrão)	ajusta para 28/02/2026
`LENIENT`	"empurra" para 03/03/2026

🏗️ Formatter sob medida com `DateTimeFormatterBuilder`

Para formatos que o `ofPattern` não resolve (partes opcionais complexas, valores padrão no parse, maiúsculas/minúsculas):

```
DateTimeFormatter flexivel = new DateTimeFormatterBuilder()
        .parseCaseInsensitive()
        .appendPattern("dd/MM/uuuu")
        .optionalStart().appendPattern(" HH:mm").optionalEnd()
        .parseDefaulting(ChronoField.HOUR_OF_DAY, 0)
        .parseDefaulting(ChronoField.MINUTE_OF_HOUR, 0)
        .toFormatter(Locale.of("pt", "BR"));

LocalDateTime.parse("27/10/2026", flexivel);        // 2026-10-27T00:00
LocalDateTime.parse("27/10/2026 16:05", flexivel);  // 2026-10-27T16:05
```

> 🚨 **Erro comum:** formatar um `LocalDate` com um padrão que tem hora (`HH:mm`) lança `UnsupportedTemporalTypeException`. O objeto precisa ter todos os campos que o padrão pede.

🏛️ Museu do Legado — conversando com o código antigo

Sistemas corporativos (bancos, principalmente) ainda têm muito `Date`, `Calendar` e `java.sql.*`. O museu tem pontes para todos eles.

Peça antiga	Problema	Vira	Ponte
`java.util.Date`	mutável, nome enganoso (é um instante)	`Instant`	`date.toInstant()` · `Date.from(instant)`
`Calendar` / `GregorianCalendar`	mês começa em 0, mutável	`ZonedDateTime`	`((GregorianCalendar) cal).toZonedDateTime()` · `GregorianCalendar.from(zdt)`
`TimeZone`	API confusa	`ZoneId`	`tz.toZoneId()` · `TimeZone.getTimeZone(zoneId)`
`SimpleDateFormat`	não é thread-safe	`DateTimeFormatter`	reescrever o padrão (as letras são quase iguais)
`java.sql.Date`	data disfarçada de instante	`LocalDate`	`sqlDate.toLocalDate()` · `java.sql.Date.valueOf(localDate)`
`java.sql.Timestamp`	idem, com nanos	`LocalDateTime` / `Instant`	`ts.toLocalDateTime()` · `Timestamp.valueOf(ldt)` · `ts.toInstant()`
`java.sql.Time`	idem, só hora	`LocalTime`	`time.toLocalTime()` · `Time.valueOf(localTime)`

```
// Date antigo → LocalDate moderno (precisa escolher o fuso!)
Date legado = new Date();
LocalDate data = legado.toInstant()
        .atZone(ZoneId.of("America/Sao_Paulo"))
        .toLocalDate();

// JDBC 4.2+: o driver já entende java.time direto 🎉
ps.setObject(1, LocalDate.of(2026, 10, 27));
LocalDate venc = rs.getObject("vencimento", LocalDate.class);
```

> 💡 Em JPA/Hibernate 5.2+ e Spring, campos `LocalDate`, `LocalDateTime`, `Instant` e `OffsetDateTime` são mapeados sem conversor. Use `java.time` nas entidades novas.

> 📚 Bairros raros: `java.time.chrono` traz calendários não-ISO (`JapaneseDate` , `HijrahDate` , `ThaiBuddhistDate` , `MinguoDate`) e `java.time.zone` guarda as regras de fuso (`ZoneRules`). Quase ninguém precisa visitá-los no dia a dia.

🎒 Cola de Bolso — qual classe eu uso?

Preciso de…	Use	Mascote
uma data sem hora (nascimento, vencimento)	`LocalDate`	📅
um horário sem data (abertura da loja)	`LocalTime`	⏰
um compromisso local (sem viajar de fuso)	`LocalDateTime`	🗓️
um evento num lugar com regras de fuso	`ZonedDateTime`	✈️
gravar em banco / mandar em API	`OffsetDateTime` ou `Instant`	📨 🤖
timestamp de log, auditoria	`Instant`	🤖
testar código que usa "agora"	`Clock`	🕰️
quanto tempo em horas/minutos	`Duration`	⏳
quanto tempo em anos/meses/dias	`Period`	🎂
total de dias/horas entre dois pontos	`ChronoUnit.X.between`	📏
competência, validade de cartão	`YearMonth`	🧩
"último dia do mês", "próxima sexta"	`TemporalAdjusters`	🪄
transformar em texto ou ler texto	`DateTimeFormatter`	🎨

🐛 Armadilhas para nunca mais cair

- [ ] **Esquecer de guardar o resultado.** `data.plusDays(1);` sozinho não faz nada — escreva `data = data.plusDays(1);` .
- [ ] **`MM` × `mm` e `yyyy` × `YYYY`** no padrão do formatter.
- [ ] **`LocalDateTime` para instantes globais.** Dois servidores em fusos diferentes vão discordar. Use `Instant` ou `OffsetDateTime` .
- [ ] **`now()` sem fuso no servidor.** Ele pega o fuso da JVM, que pode ser UTC. Prefira `LocalDate.now(ZoneId.of("America/Sao_Paulo"))` ou um `Clock` injetado.
- [ ] **Comparar com `equals` entre fusos.** Dois `ZonedDateTime` do mesmo instante em fusos diferentes não são `equals` ; use `isEqual()` .

- [ ] **`Period.getDays()` como total.** É só a sobra; para o total use `ChronoUnit.DAYS.between`.
- [ ] **Somar mês no fim do mês.** `LocalDate.of(2026, 1, 31).plusMonths(1)` dá 28/02, e voltar com `minusMonths(1)` dá 28/01, não 31/01.
- [ ] **Formatter sem `Locale`** imprimindo mês em inglês no servidor.

⚡ Exceptions que você vai encontrar

Exception	Quando aparece
`DateTimeParseException`	o texto não bate com o padrão
`DateTimeException`	valor impossível (`LocalDate.of(2026, 2, 30)`) ou fuso desconhecido
`UnsupportedTemporalTypeException`	pediu um campo que o objeto não tem (hora num `LocalDate`)
`ZoneRulesException`	`ZoneId.of("Brasil/Brasilia")` — id de fuso inexistente